FLUTTER -05

LAB ASSIGMENTS

G-TEC EDUCATION

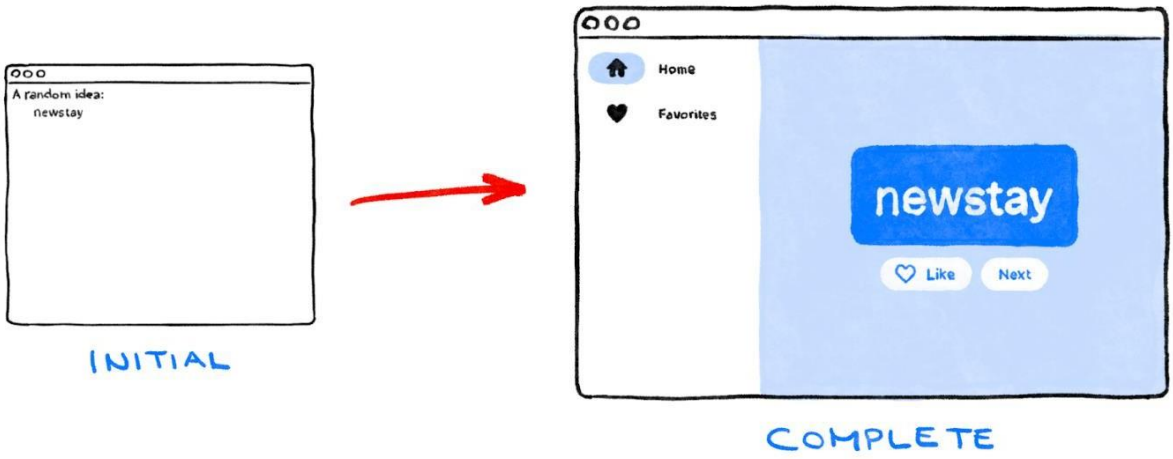
— G-TEC Group of Institutions —

www.gteceducation.com

Admin Office House of G-TEC, Calicut-02., India. | Corp. Office Peace Centre, Singapore – 228149

FINAL

You took a non-functional scaffold with a Column and two Text widgets, and made it into a responsive, delightful little app.



What we've covered

- The basics of how Flutter works
- Creating layouts in Flutter
- Connecting user interactions (like button presses) to app behavior
- Keeping your Flutter code organized
- Making your app responsive
- Achieving a consistent look & feel of your
- Experiment more with the app you wrote during this lab.
- Look at the code of this advanced version of the same app, to see how you can add animated lists, gradients, cross-fades, and more.

```
// ignore_for_file: prefer_const_constructors, prefer_const_literals_to_create_immutables

import 'package:english_words/english_words.dart'; import
'package:flutter/material.dart';
import 'package:provider/provider.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) { return
    ChangeNotifierProvider(
      create: (context) => MyAppState(),
      child: MaterialApp(
        title: 'Namer App', theme:
        ThemeData(
        useMaterial3: true,
          colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepOrange),
        ),
        home: MyHomePage(),
      ),
    );
```

```
 }
}

class MyAppState extends ChangeNotifier { var
 current = WordPair.random();
 var history = <WordPair>[];

 GlobalKey? historyListKey;

 void getNext() {
  history.insert(0, current);
  var animatedList = historyListKey?.currentState as AnimatedListState?;
  animatedList?.insertItem(0);
  current = WordPair.random();
  notifyListeners();
 }

 var favorites = <WordPair>[];

 void toggleFavorite([WordPair? pair]) { pair
  = pair ?? current;
  if (favorites.contains(pair)) {
   favorites.remove(pair);
  } else {
   favorites.add(pair);
  }
```

```
    notifyListeners();
  }

  void removeFavorite(WordPair pair) {
    favorites.remove(pair);
    notifyListeners();
  }
}

class MyHomePage extends StatefulWidget { @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> { var
  selectedIndex = 0;

  @override
  Widget build(BuildContext context) {
    var colorScheme = Theme.of(context).colorScheme;

    Widget page;
    switch (selectedIndex) { case
      0:
        page = GeneratorPage();
        break;
```

```
  case 1:
    page = FavoritesPage();
    break;
  default:
    throw UnimplementedError('no widget for $selectedIndex');
}

// The container for the current page, with its background color
// and subtle switching animation. var
mainArea = ColoredBox(
  color: colorScheme.surfaceVariant,
  child: AnimatedSwitcher(
    duration: Duration(milliseconds: 200),
    child: page,
  ),
);

return Scaffold(
  body: LayoutBuilder(
    builder: (context, constraints) {
      if (constraints.maxWidth < 450) {
        // Use a more mobile-friendly layout with BottomNavigationBar
        // on narrow screens.
        return Column(
          children: [
            Expanded(child: mainArea),
```

```
        SafeArea(
          child: BottomNavigationBar(
            items: [
              BottomNavigationBarItem(
                icon: Icon(Icons.home),
                label: 'Home',
              ),
              BottomNavigationBarItem(
                icon:      Icon(Icons.favorite),
                label: 'Favorites',
              ),
            ],
            currentIndex: selectedIndex,
            onTap: (value) {
              setState(() {
                selectedIndex = value;
              });
            },
          ),
        )
      ],
    );
  } else { return
    Row(
    children: [
      SafeArea(
```

```
          child: NavigationRail(
            extended: constraints.maxWidth >= 600,
            destinations: [
              NavigationRailDestination(
                icon: Icon(Icons.home),
                label: Text('Home'),
              ),
              NavigationRailDestination(
                icon:      Icon(Icons.favorite),
                label: Text('Favorites'),
              ),
            ],
            selectedIndex: selectedIndex,
            onDestinationSelected: (value) { setState(() {
              selectedIndex = value;
            });
            },
          ),
        ),
        Expanded(child: mainArea),
      ],
    );
  }
 },
),
```

```
  );
 }
}

class GeneratorPage extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
  var appState = context.watch<MyAppState>(); var
  pair = appState.current;

  IconData icon;
  if (appState.favorites.contains(pair)) {
   icon = Icons.favorite;
  } else {
   icon = Icons.favorite_border;
  }

  return Center(
   child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
     Expanded(
      flex: 3,
      child: HistoryListView(),
     ),
     SizedBox(height: 10),
```

```
      BigCard(pair: pair),
      SizedBox(height: 10),
      Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          ElevatedButton.icon(
            onPressed: () {
              appState.toggleFavorite();
            },
            icon: Icon(icon),
            label: Text('Like'),
          ),
          SizedBox(width: 10),
          ElevatedButton(
          onPressed: () {
              appState.getNext();
            },
            child: Text('Next'),
          ),
        ],
      ),
      Spacer(flex: 2),
    ],
  ),
 );
}
```

```
}

class BigCard extends StatelessWidget {
  const BigCard({
    Key? key,
    required this.pair,
  }) : super(key: key);

  final WordPair pair;

  @override
  Widget build(BuildContext context) { var
    theme = Theme.of(context);
    var style = theme.textTheme.displayMedium!.copyWith( color:
      theme.colorScheme.onPrimary,
    );

    return Card(
      color: theme.colorScheme.primary,
      child: Padding(
        padding: const EdgeInsets.all(20),
        child: AnimatedSize(
          duration: Duration(milliseconds: 200),
          // Make sure that the compound word wraps correctly when the window
          // is too narrow.
          child: MergeSemantics(
```

```
          child: Wrap(
            children: [
            Text(
                pair.first,
                style: style.copyWith(fontWeight: FontWeight.w200),
              ),
              Text(
                pair.second,
                style: style.copyWith(fontWeight: FontWeight.bold),
              )
            ],
          ),
        ),
      ),
    ),
  );
 }
}

class FavoritesPage extends StatelessWidget { @override
 Widget build(BuildContext context) { var
   theme = Theme.of(context);
   var appState = context.watch<MyAppState>();

   if (appState.favorites.isEmpty) {
```

```
  return Center(
    child: Text('No favorites yet.'),
  );
}

return Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Padding(
      padding: const EdgeInsets.all(30),
      child: Text('You have '
          '${appState.favorites.length} favorites:'),
    ),
    Expanded(
      // Make better use of wide windows with a grid. child:
      GridView(
        gridDelegate: SliverGridDelegateWithMaxCrossAxisExtent(
          maxCrossAxisExtent: 400,
          childAspectRatio: 400 / 80,
        ),
        children: [
          for (var pair in appState.favorites) ListTile(
              leading: IconButton(
                icon: Icon(Icons.delete_outline, semanticLabel: 'Delete'), color:
                theme.colorScheme.primary,
```

```
                onPressed: () {
                  appState.removeFavorite(pair);
                },
              ),
              title: Text(
                pair.asLowerCase,
                semanticsLabel: pair.asPascalCase,
              ),
            ),
          ],
        ),
      ),
    ],
  );
 }
}
class HistoryListView extends StatefulWidget {
 const HistoryListView({Key? key}) : super(key: key);

 @override
 State<HistoryListView> createState() => _HistoryListViewState();
}

class _HistoryListViewState extends State<HistoryListView> {
 /// Needed so that [MyAppState] can tell [AnimatedList] below to animate
```

```
/// new items.
final _key = GlobalKey();

/// Used to "fade out" the history items at the top, to suggest continuation. static
const Gradient _maskingGradient = LinearGradient(
  // This gradient goes from fully transparent to fully opaque black... colors:
  [Colors.transparent, Colors.black],
  // ... from the top (transparent) to half (0.5) of the way to the bottom. stops: [0.0,
  0.5],
  begin: Alignment.topCenter,
  end: Alignment.bottomCenter,
);

@override
Widget build(BuildContext context) {
  final appState = context.watch<MyAppState>();
  appState.historyListKey = _key;

  return ShaderMask(
    shaderCallback: (bounds) => _maskingGradient.createShader(bounds),
    // This blend mode takes the opacity of the shader (i.e. our gradient)
    // and applies it to the destination (i.e. our animated list). blendMode:
    BlendMode.dstIn,
    child: AnimatedList(
      key: _key,
      reverse: true,
```

```
      padding: EdgeInsets.only(top: 100),
      initialItemCount: appState.history.length,
      itemBuilder: (context, index, animation) { final
      pair = appState.history[index]; return
      SizeTransition(
        sizeFactor: animation,
        child: Center(
          child: TextButton.icon( onPressed:
            () {
              appState.toggleFavorite(pair);
            },
            icon: appState.favorites.contains(pair)
                ? Icon(Icons.favorite, size: 12)
                : SizedBox(),
            label: Text(
              pair.asLowerCase,
              semanticsLabel: pair.asPascalCase,
            ),
          ),
        ),
      );
    },
  ),
 );
 }
}
```